

Hospital Management System (HMS) – Project Report

Name: Ashrita Khanta

Roll Number: 22f1000828

Email: 22f1000828@ds.study.iitm.ac.in

About Me : I am currently in diploma level of IITM BS Degree in Data Science & application.

Description : This project implements a hospital management system that digitizes core workflows such as patient registration, doctor onboarding, appointment booking, and treatment recording, with separate interfaces for Admin, Doctor, and Patient roles. The system supports role-based dashboards, medical history tracking, scheduled email reminders, monthly reports for doctors, CSV export of treatment history, and Redis-based caching to improve performance.

AI/LLM Usage : AI/LLM assistance was used for approximately 15% of the work for code optimization, debugging specific issues, and documentation formatting. Primary development, architecture design, and feature implementation were done independently.

Technologies Used:

- Flask – Web framework for routing, views, and template rendering.
- SQLAlchemy – ORM for defining models such as User, Doctor, Patient, Appointment, Treatment, Department, and DoctorAvailability.
- Flask-Login – User session and authentication management with role-based access control.
- Flask-WTF, WTForms – Form handling, validation, and CSRF protection.
- SQLite – Default relational database; can be replaced with PostgreSQL/MySQL.
- Celery – Background task queue for reminders, monthly reports, and CSV exports.
- Redis – Message broker, Celery backend, and caching layer.
- Flask-Mail – Email sending for notifications and reports.
- Flask-Caching – Redis-based caching of frequently accessed data.
- Pandas – CSV export generation from treatment data.
- Bootstrap 5 - CSS framework for responsive design and UI components
- HTML5/CSS3 - Frontend markup and styling

Purpose: Flask and SQLAlchemy provide the core backend and data layer, Flask-Login and Flask-WTF secure user access, Celery and Redis enable scalable background processing, while Flask-Mail, Pandas, and WeasyPrint handle communication, exports, and reporting.

System Design

Database Schema

The system uses a normalized schema with clear one-to-many and many-to-many relationships.

- User: Base user accounts with roles: Admin, Doctor, Patient, plus status and login details.
- Doctor: Doctor profile linked to User; stores specialization, department, and contact info.
- Patient: Patient profile linked to User; stores personal, contact, and emergency contact details.
- Department: Master table for hospital departments such as Cardiology, Neurology, Orthopedics, Pediatrics, Dermatology, and General Medicine, initialized via script.
- DoctorAvailability: Availability slots for each doctor, used for appointment booking.
- Appointment: Links doctor and patient with date, time, and status (Booked, Completed, Cancelled), with constraints to prevent double booking for a given doctor and time.
- Treatment: Diagnosis, prescription, and notes associated with a completed appointment, powering patient medical history.

Design rationale: Separating User, Doctor, Patient, Appointment, and Treatment improves clarity, enforces referential integrity, and simplifies both operational queries and reporting.

Architecture and Modules:

The project follows a modular Flask blueprint architecture.

- app/
 - init.py – Flask app factory wiring config, DB, login manager, cache, and Celery integration.
 - models.py – All ORM models and relationships.
 - admin/ – Admin blueprint (routes, forms, templates) for managing doctors, patients, and appointments.

- doctor/ – Doctor blueprint for viewing schedules, updating availability, and entering treatments.
- patient/ – Patient blueprint for registration, booking, and history views.
- auth/ – Authentication blueprint for login, logout, and registration.
- main/ – General pages and landing routes.
- config.py – Central configuration (database URL, Redis URLs, Celery settings, mail server, caching options, session lifetime).
- init_admin.py – One-time initialization of database, default admin user, and default departments.
- celery_app.py – Celery instance and periodic task scheduling (worker + beat).
- app.py – Entry script creating and running the Flask app on port 5000.

Key Features by Role:

Admin:

- Login to an admin dashboard showing statistics for patients, doctors, and appointments.
- Add and update doctor profiles, assign departments, and deactivate/blacklist accounts when needed.
- View and search patients, inspect complete medical history (appointments and treatments), and manage all appointments.

Doctor:

- View a dashboard of upcoming appointments for the current day and week.
- Mark appointments as Completed or Cancelled and enter diagnosis, prescriptions, and notes after consultations.
- Manage availability for the next 7 days and view patient history for continuity of care.

Patient:

- Self-register, manage profile, and login to a patient dashboard.
- Search for doctors by specialization or name and book appointments only in available slots.
- Reschedule or cancel appointments, view past visits and treatment details, and request CSV export of treatment history.

Background Jobs and Caching:

Celery Tasks:

- Daily Reminders:
 - Runs every day at 08:00 AM IST via Celery Beat and sends email reminders to patients with same-day appointments.
- Monthly Doctor Reports:
 - Runs on the 1st of each month at 09:00 AM IST to generate HTML and PDF reports per doctor, covering appointment counts, statuses, treatments, and diagnosis summary.
 - Emails reports to doctors and stores them under instance/reports.
- CSV Export:
 - Triggered by patients; exports complete treatment history to CSV using Pandas and stores files in instance/exports, with an email notification when ready.

Caching Strategy:

Redis caching (via Flask-Caching) is applied to reduce latency and database load.

- Cached data: doctor availability (30–60 minutes), patient search results (~5–10 minutes), doctor search (~10 minutes), and dashboard aggregates (~5 minutes).
- Cache invalidation: on appointment create/update/cancel, doctor availability changes, or updates to patient/doctor profiles, ensuring consistency.

Video: https://drive.google.com/file/d/1lxwVTFGJ79BkD7-Q8fyC2fnvOJ1w8Ta3/view?usp=drive_link